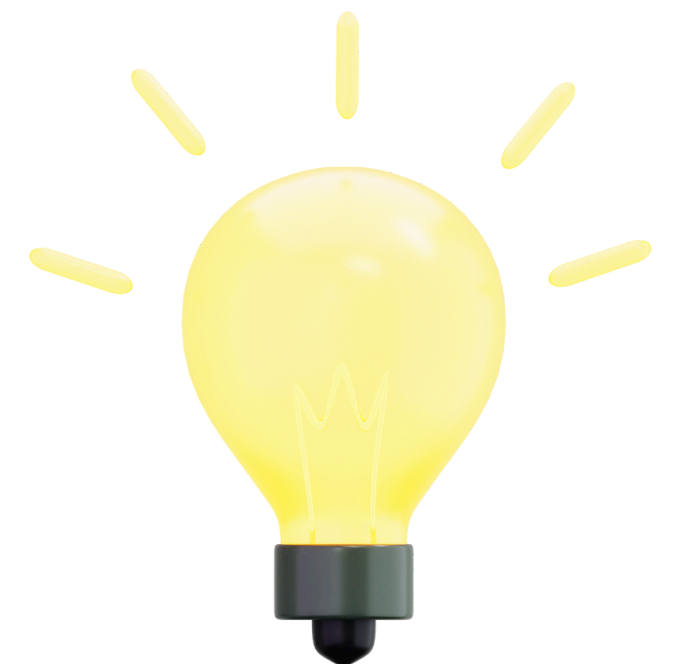




CO-OPERATIVE INTERNSHIP FINAL PRESENTATION

presented by Sirisuk Tharntham



Contents

- **Introduction**
- **My project during internship**
- **Summarize the learning outcomes**
- **Further improvements**

Introduction



Sirisuk Tharntham (PUN)

student id 6610402230

A 4th year student from Computer Science,
Kasetsart University, Thailand.

- come to University of Passsau, Germany
- For CO-Operative Internship
- at Fakultät für Informatik und Mathematik
from 16 May - 11 Augs (87 days)
- CO-Advisor
 - Prof. Dr.-Ing. Christian Hammer
 - Dr. Mohammad Rezaalipour

Project

Description

the project focus on the static analysis field both theoretical and practical implementations

- base on static analysis framework call “WALA”
- with specify implementation on JAVA language
- our project have 5 phases that break into 3 parts

Project cont.

3 Parts of project

WALA getting start &
practical implementations

Static analysis techniques
research

Tools implementation

Part 1 WALA practical implementation

My responsibilities

1. Discovering and try to understand WALA framework
2. Run the WALA starting source code to understand WALA structures
3. Develop shell script to initialize and automate CI/CD and fix JAVA compatibilities for the framework
4. Develop reproducible source code and documentation
5. project manager for this part

WALA topologies

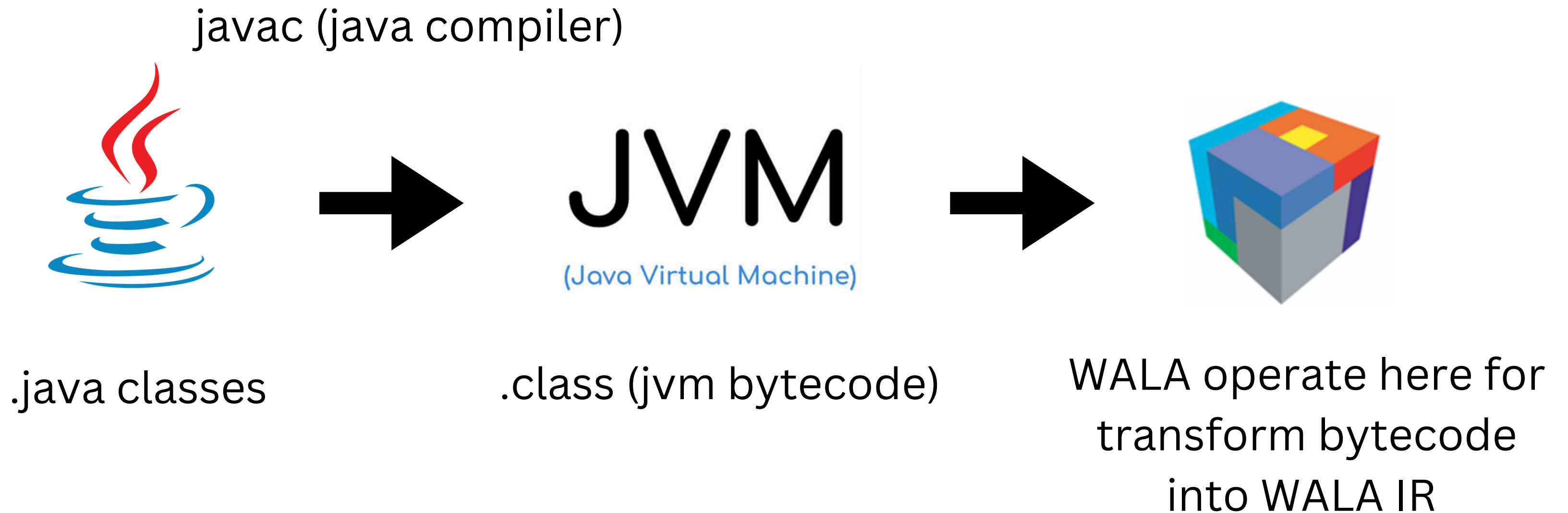


WALA

T. J. WATSON LIBRARIES FOR ANALYSIS

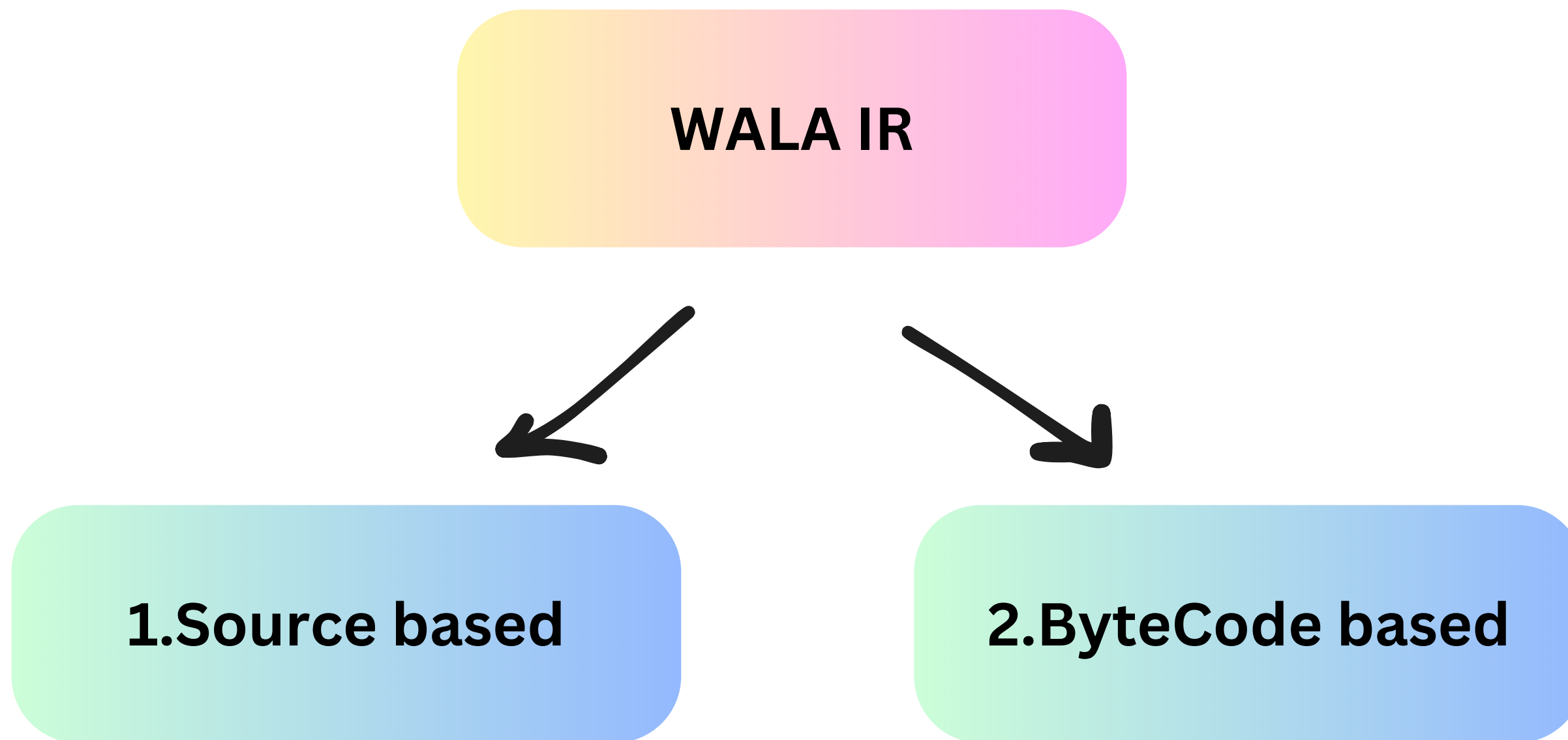
WALA topologies cont.

WALA framework provide static analysis capabilities for Java bytecode



WALA topologies cont.

Since WALA operated on ByteCode, byconstructing WALA IR can be devided into 2 pathway



WALA topologies cont.

1.Source based

.java classes

Eclipse Java Compiler

cast IR (from in-memory tree)

ClassHierachy

2.ByteCode based

.class ByteCode

WALA IR

Making runnable WALA source code

Once We have WALA IR we can construct the CFG for more analysis.
The framwork provide startting code samples with following structure.

```
WALA-start/
├── build.gradle.kts          ← Gradle build config
├── settings.gradle.kts      ← Gradle project name
├── gradlew / gradlew.bat    ← Gradle wrapper (no install needed)
├── run_analysis.sh          ← main entry point: builds stdlib cache + runs a driver
├── scope.txt                ← auto-generated WALA scope file
├── src/
│   └── main/
│       ├── java/com/ibm/wala/examples/
│       │   ├── drivers/          ← runnable analysis entry points ✓
│       │   ├── analysis/          ← supporting analysis implementations
│       │   ├── analysisscope/     ← AnalysisScope setup example
│       │   └── util/              ← shared helpers
│       └── resources/
│           ├── wala.properties    ← WALA config (stdlib path, output dir)
│           ├── Exclusions.txt      ← class patterns WALA should skip
│           └── test-files/
│               └── fibo.js          ← sample JS target
└── out/                        ← generated output (PDF, dot, log)
```

I created testsample
fibo.java/.js
with DP fibonacci
code for me and my
friend tests

fixing compatiibilities

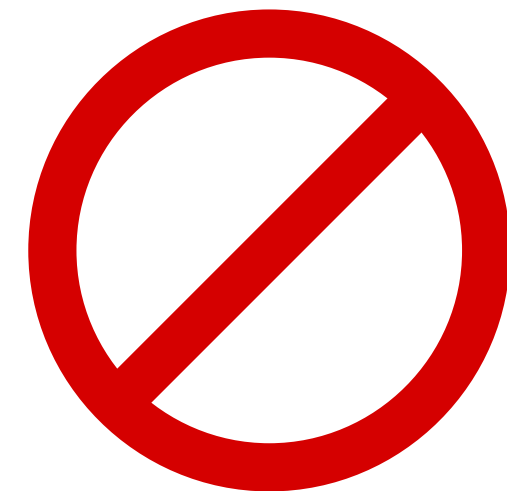
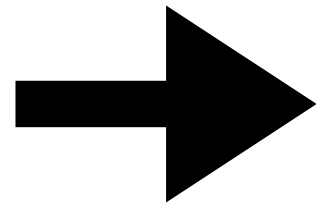
When I tried to run the driver source code there are lots of compatibilities issue at least I should fix it before it can be run.

- Each driver have specific arguments
- Gradle, wala.properties and Exclusions config files
- ***Dependencies and Standard Library requirements***

fixing compatiibilities cont.

Because WALA use **JAVA standard library as requiemennts** then there is problems, I found that WALA load java.base in JAVA_HOME that missing in newer JDK version (> jdk 8)

At JAVA_HOME



WALA try to load standard library

jdk 8
java.base
(jre/lib/rt.jar)

jdk >8
missing

fixing compatiibilities cont.

In newer version of jdk, stand lib change from monolithic into modular base for better performance since monolithic load all lib but the later use only need module

Era	Format	Layout
Java $\leq$ 8	<code>rt.jar</code> (~60 MB monolithic JAR)	<code>jre/lib/rt.jar</code> — entire stdlib in one file
Java 9+	<code>.jmod</code> files (~70 named modules)	<code>jmods/java.base.jmod</code> , <code>java.sql.jmod</code> , ...

SO WALA NOT FOUND STDLIB

I have tried searching and the develop
some workarround that should work to fix this problem



fixing compatiibilities cont.

The Fix

```
unzip -q "$JAVA_HOME/jmods/java.base.jmod" -d /tmp/wala-stdlib/x || true  
# Repack classes/ as a standard JAR  
jar cf /tmp/wala-stdlib/java.base.jar -C /tmp/wala-stdlib/x/classes .
```

I have found way to fix that java provide cli to unpack .jmod then repack into .jar that WALA can read

```
java_runtime_dir=/tmp/wala-stdlib
```

then writedown the config file to let the framework know where is the standard library so this fix work well in newer version

CI/CD scripts for reproducible development

Now I can test those driver with all controll variables (e.g. the config files , JAVA_HOME, the standard library and program arguments)

- To satisfy those variables everytine run the program cause lack of developement then I decide to write the automation scripts

```
BUILD_DIR="$(cd "$SCRIPT_DIR" && realpath "/absolute/path/to/buildpath")"
# --- WALA properties ---
mkdir -p src/main/resources/
echo "java_runtime_dir=/tmp/wala-stdlib" > src/main/resources/wala.properties

# --- ScopeFile ---
SCOPE_FILE="$SCRIPT_DIR/scope.txt"

{
echo "Primordial,Java,stdlib,none"
echo "Application,Java,binaryDir,$BUILD_DIR"
} > "$SCOPE_FILE"

JDK_HOME="${JAVA_HOME:?JAVA_HOME must be set}"
```

program config

```
if [ ! -f /tmp/wala-stdlib/java.base.jar ]; then
echo "Building WALA stdlib cache (one-time)..."
mkdir -p /tmp/wala-stdlib/x
unzip -q "$JDK_HOME/jmods/java.base.jmod" -d /tmp/wala-stdlib/x || true
"$JDK_HOME/bin/jar" cf /tmp/wala-stdlib/java.base.jar \
-C /tmp/wala-stdlib/x/classes .
rm -rf /tmp/wala-stdlib/x
fi

# Test script below this section
# --- ScopeFileCallGraph ---
"$SCRIPT_DIR/gradlew" -p "$SCRIPT_DIR" run \
-PmainClass=com.ibm.wala.examples.drivers.ScopeFileCallGraph \
--args="-scopeFile $SCOPE_FILE -mainClass Lcom/example/Main"
```

build requirement std lib

CI/CD scripts for reproducible development

those scripts help development workflow and since I have to test the driver and reproduce test in many jdk variance so

- the version controll is nessesary (to fix)
- and there is also the problem that when continue running gradle instance may not reinitiate, so this cause stuck and wrong jdk version (to fix)

```
#JAVA_HOME="/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home"
JAVA_VERSION=$(("$JAVA_HOME/bin/java" --version | awk 'NR==1{split($2,a,"."); print a[1]}')
sed -i '' "s/JavaLanguageVersion\\.of\\([0-9]*\\.\\.*\\)/JavaLanguageVersion.of($JAVA_VERSION)/" "$SCRIPT_DIR/build.gradle.kts"
echo "=== build.gradle.kts toolchain set to Java $JAVA_VERSION ==="

"$SCRIPT_DIR/gradlew" -p "$SCRIPT_DIR" --stop
"$SCRIPT_DIR/gradlew" -p "$SCRIPT_DIR" --version | grep JVM
"$SCRIPT_DIR/gradlew" -p "$SCRIPT_DIR" clean compileJava
```

I introduce scripts that integrate with text substitution program to help change version controll and verification script for runner to get fresh instance

The driver testing and documentation

ScopeFileCallGraph

Table of Contents

- Functionality
- Input & Output
- Run Instructions
 - Prerequisites
 - Dependencies
 - Steps
 - Class Name Format
 - Program Code
 - Sample Input
 - Expected Output
- Reproducibility

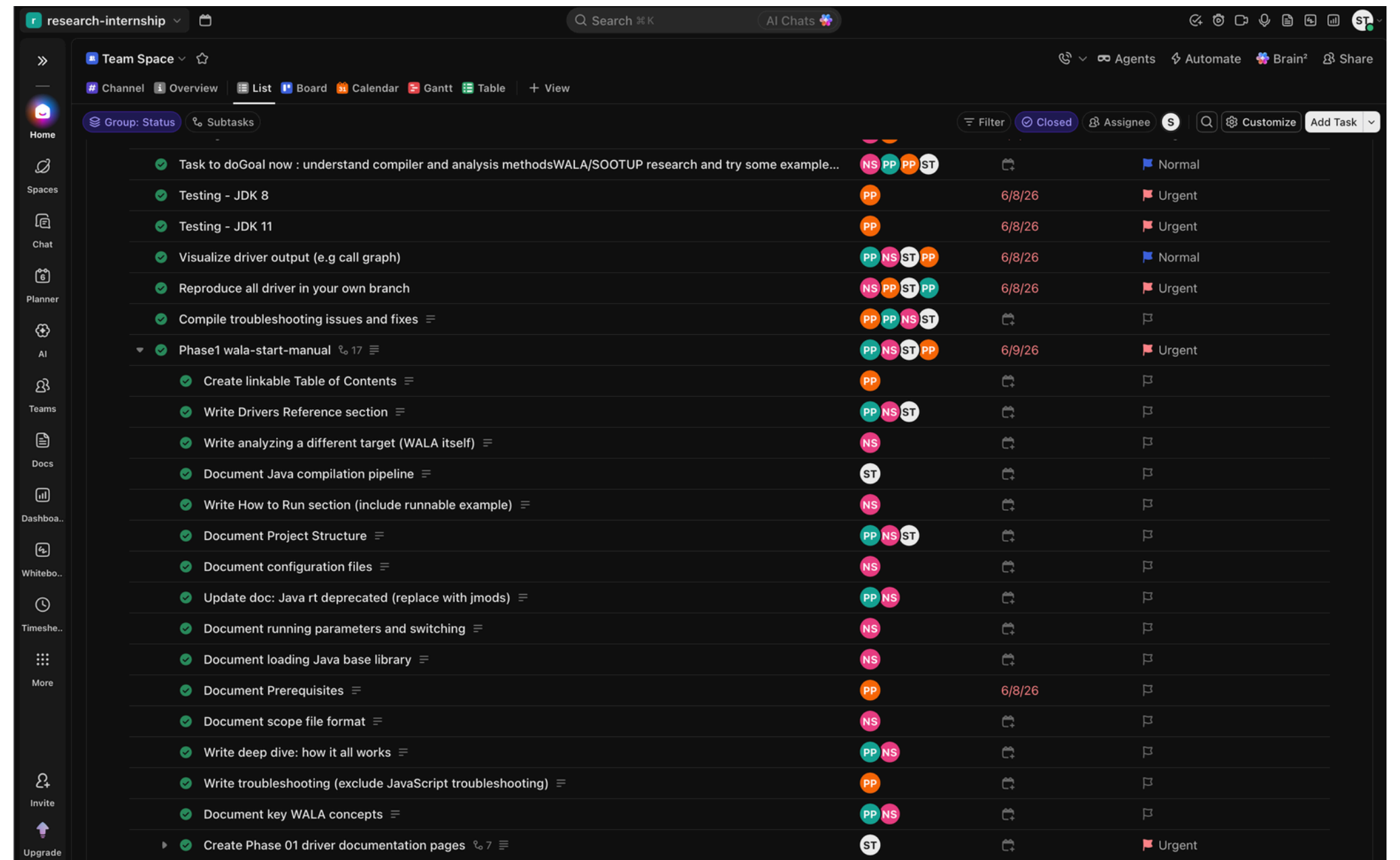
I bring my script to test the ScopeFileCallGraph driver with my DP fibonanci code and try reproduce in jdk variance.

- the driver read the scope file to initialize WALA analysis program
- from Java Classes to build call graph display in terminal std ouput

https://github.com/punsnx/coop-internship-2026/blob/sirisuk/main/phase_01/ScopeFileCallGraph.md

Team project management

In the first part of our project I work in team and I also be the project manager to manage works and distribute works to team member by using Agile principle with Tools call “clickup” (similar to JIRA KANBAN board)



Part 2 Static Techniques research

My responsibilities

1. Discover new static analysis techniques
2. Verifying that the each technique have or not have implement within compiler or static analysis tool (e.g. Sonar qube)

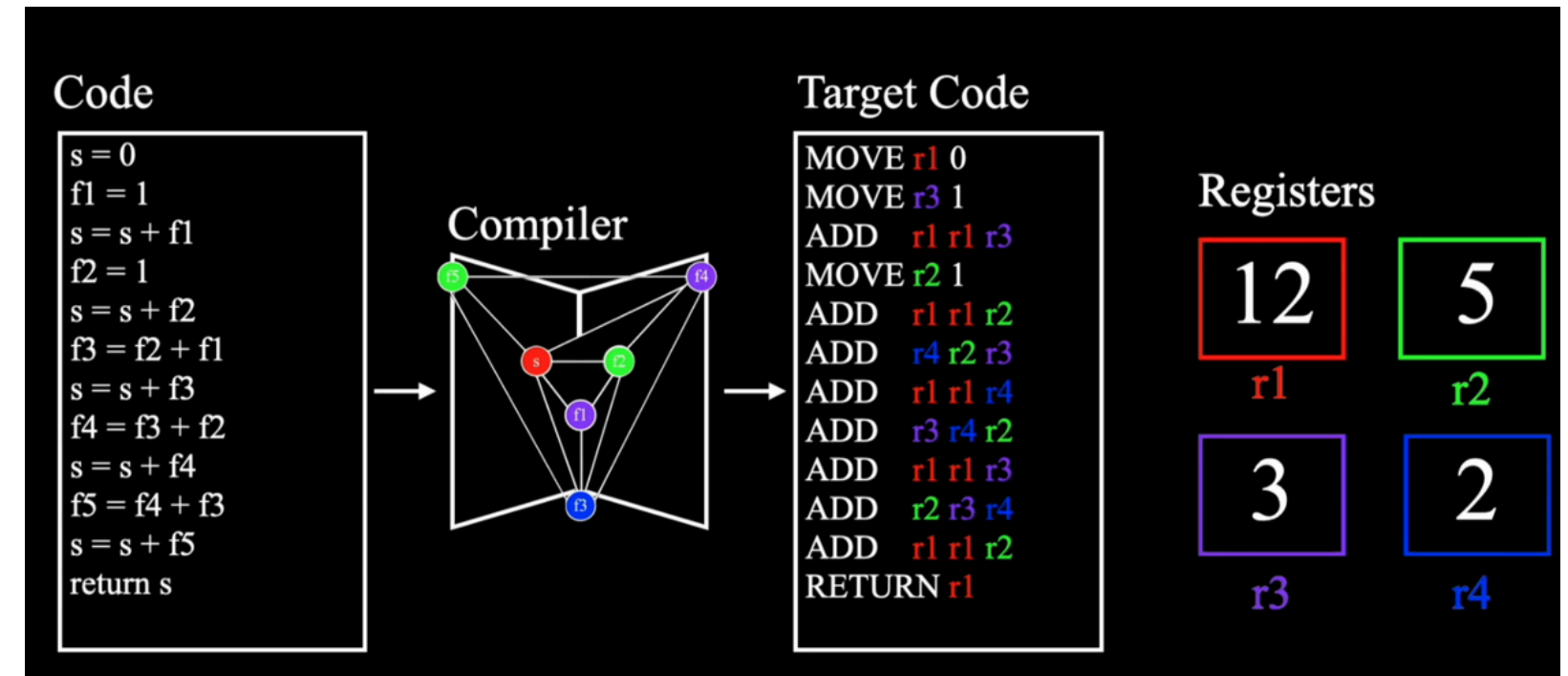
Techniques discovering

In this part I have to research for new interesting static techniques and check that those techniques implement with in compiler/WALA or not. Following are list of my researched techniques

- Techniques
 - Graph Register Allocation
 - Strength Reduction
 - Interval / Range Analysis
 - May-Must Dual Analysis
 - Integer Linear Programming (ILP)
 - Omega Test

Graph Register Allocation

use an idea of graph coloring algorithm
to perform register allocation
(in compiler allocation part)



The screenshot shows a code editor with two C functions and their corresponding assembly code. The first function, `is_odd`, uses a modulo operation to check if a value is odd. The second function, `remainder`, uses a modulo operation to calculate the remainder of a division. The assembly code for `is_odd` uses the `mov`, `and`, and `ret` instructions. The assembly code for `remainder` uses the `mov`, `cdq`, `idiv`, and `ret` instructions.

```
1 bool is_odd(int val) {
2     return val % 2;
3 }

1 int remainder(int val, int val2) {
2     return val % val2;
3 }
```

```
1 is_odd(int):
2     mov     eax, edi
3     and     eax, 1
4     ret

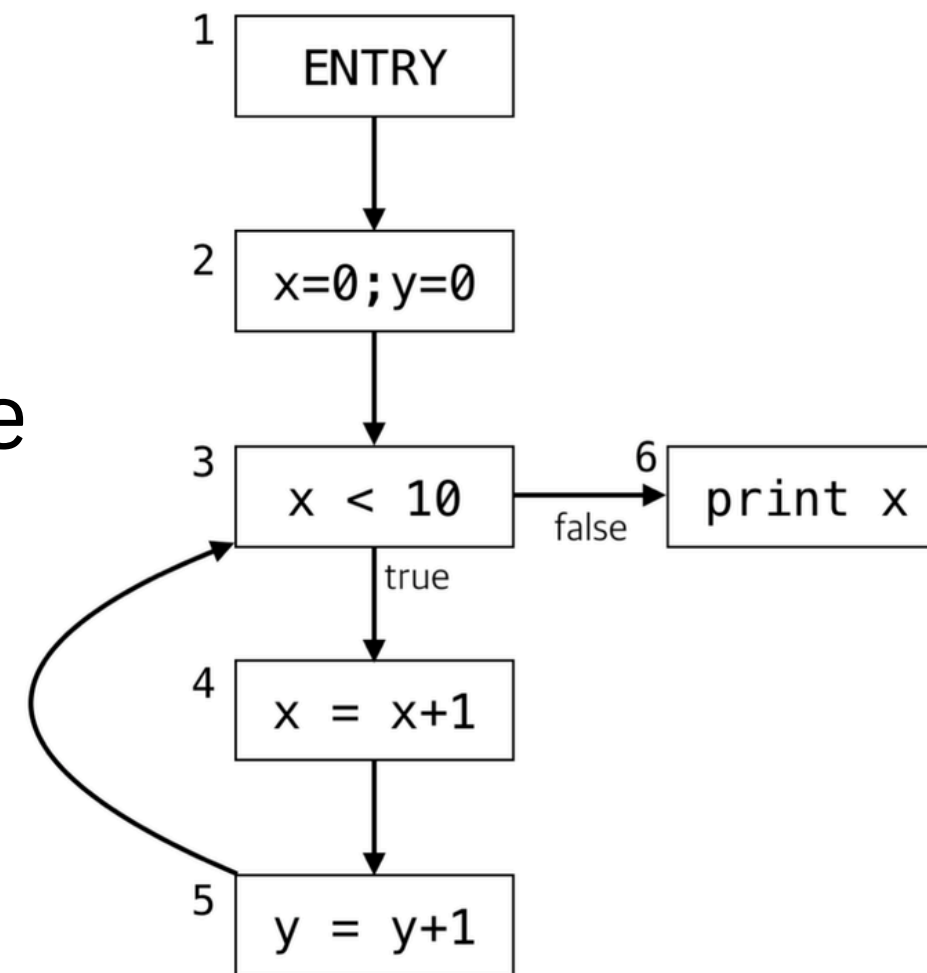
1 remainder(int, int):
2     mov     eax, edi
3     cdq
4     idiv    esi
5     mov     eax, edx
6     ret
```

Strength Reduction

the technique to reduce expensive operation (e.g modulo to bitwise)

Interval / Range Analysis

the technique to track bound value at each program point and compute the possible range [lo, hi] that each variable can hold.



Node	Result
1	$x \mapsto \perp$ $y \mapsto \perp$
2	$x \mapsto [0, 0]$ $y \mapsto [0, 0]$
3	$x \mapsto [0, 9]$ $y \mapsto [0, +\infty]$
4	$x \mapsto [1, 10]$ $y \mapsto [0, +\infty]$
5	$x \mapsto [1, 10]$ $y \mapsto [1, +\infty]$
6	$x \mapsto [10, 10]$ $y \mapsto [0, +\infty]$

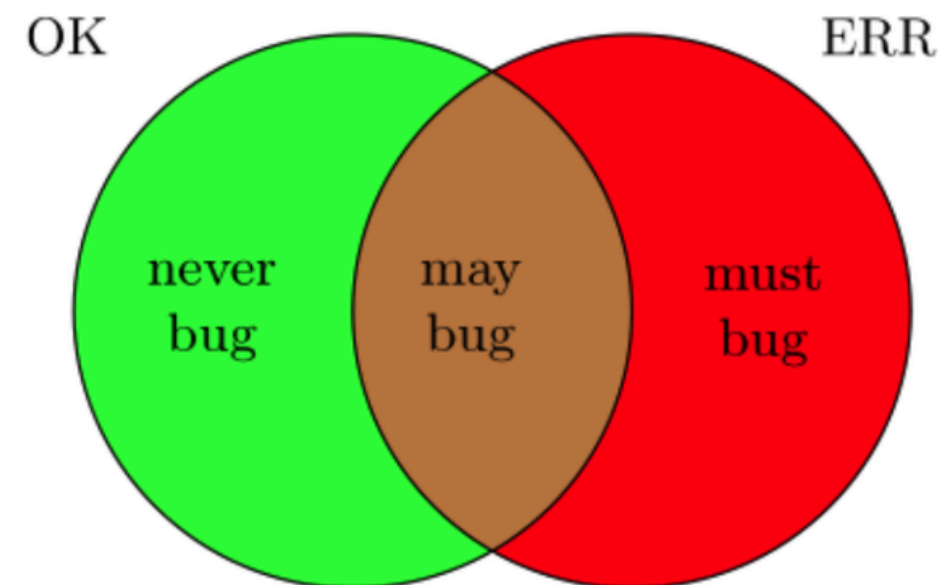


Fig. 1. Classifying bugs.

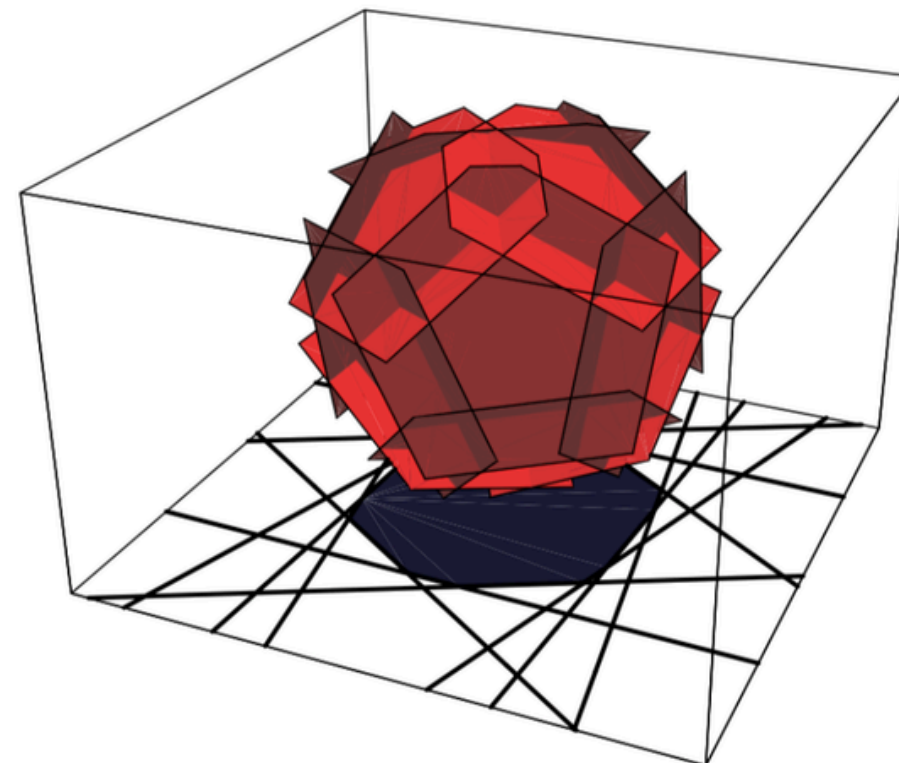
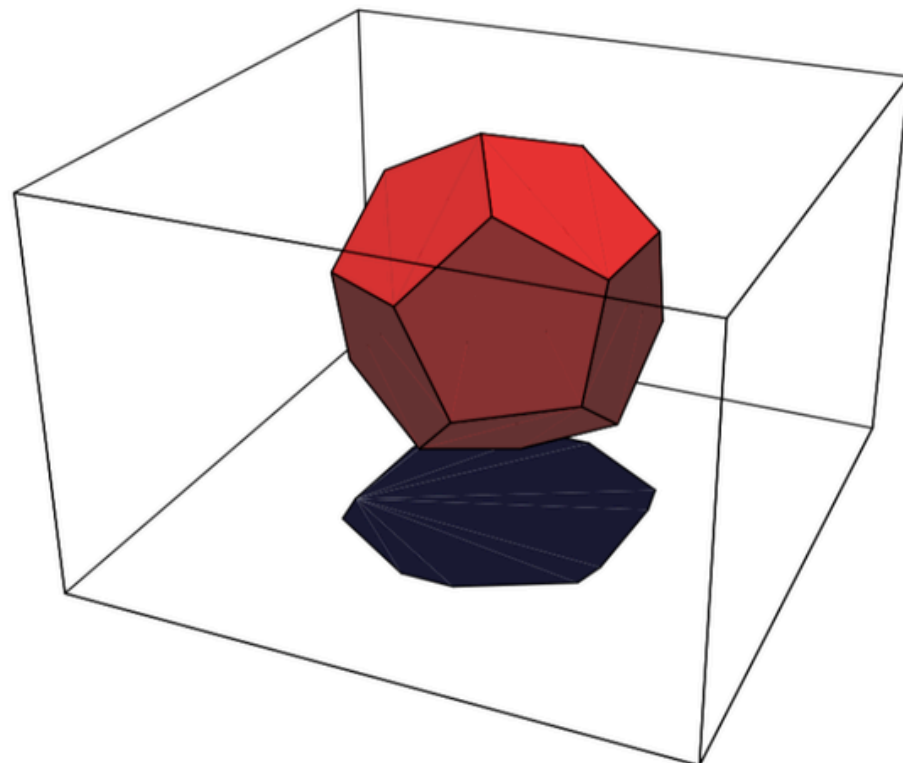
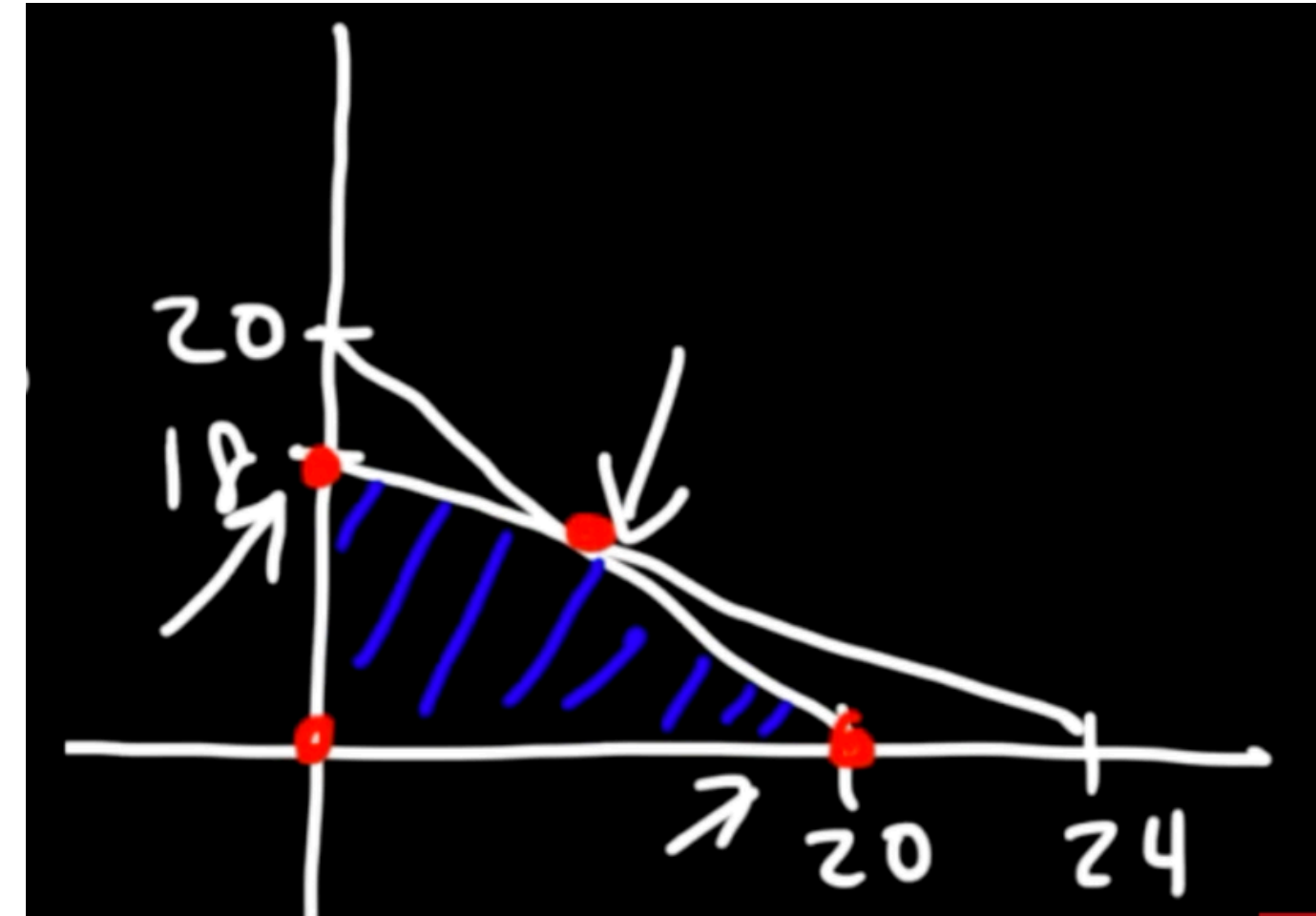
May-Must Dual Analysis

parallel analyses to find bugs and prove safety.

- a may (over-approximation)
- a must (under-approximation)
- e.g dead store variable
 - must - if X overwritten B after A then safe to remove otherwise A may still live

Integer Linear Programming (ILP)

Formulate a compiler optimization problem as a system of linear constraints and integer variables, then solve it to find the provably optimal solution.



Omega Test

the decision procedure to test whether ILP constrain have solution or not
e.g formulate for loop in parallel with bound constrains
testing race condition
if no solution then race free

Part 3 Tools implementation

My responsibilities

1. Working on the Control flow Graph (CFG) construction with WALA framework
2. Discovering for relation between WALA IR and ByteCode
3. Transform WALA IR into JAVA source code
4. Develop merging algorithm to find grouping and merge each CFG'
Basic Block

Control Flow Graph (CFG) generation via WALA

Goal

by this final part I have to create tool for generating the control flow graph (CFG) with human friendly contex.

work

from the knowledge that I have done by discovering WALA basics help me to understand the program workflow

- I start build the tools from scratch with basics init module.

```
public static void main(String[] args) throws IOException, CallG
    long start = System.currentTimeMillis();

    AnalysisScope scope = readScope(args);
    setExclusion(scope);
    IClassHierarchy cha = ClassHierarchyFactory.make(scope);

    System.out.println(cha.getNumberOfClasses() + " classes");
    System.out.println(Warnings.asString());
    Warnings.clear();

    AnalysisOptions options = setAnalysisOptions(cha);
    AnalysisCache cache = new AnalysisCacheImpl();
    CallGraph cg = buildCallGraph(options, cache, cha);

    SimpleVisualizer.printAppCallgraph(cg);
    SimpleVisualizer.printCFG(cg);
    NewGraphVisualizer.printMergedCFG(cg, sourceDir);
    PrintPDFVisualze.printPDF(cg, sourceDir);
    GraphVisualizer.printMergedCFG(cg, sourceDir);
    long end = System.currentTimeMillis();
    System.out.println("done");
    System.out.println("took " + (end - start) + "ms");
    System.out.println(CallGraphStats.getStats(cg));
```

Getting to know the flow

by starting loop through all node
from call graph (methods) each
have CFG basic block

- I tried to printout by debugging
std output
- and logging the next sucessor
BasicBlock

```
public class SimpleVisualizer { 4 usages 2 punsnx

    public static void printCFG(CallGraph cg) { no usages 2 punsnx
        for (CGNode node : cg) {
            IR ir = node.getIR();
            if (!isAppMethod(node) || ir == null) continue;

            System.out.println("\n=== " + node.getMethod().getSignature() + " ===");

            SSACFG cfg = ir.getControlFlowGraph();
            for (ISSABasicBlock bb : cfg) {
                printBasicBlock(bb, cfg);
            }
        }
    }
}
```

```
[com.sirisuk.Main.main([Ljava/lang/String;)V]
--> [app] com.sirisuk.AnalysisClass.run()V

[com.sirisuk.AnalysisClass.run()V]
--> [app] com.sirisuk.AnalysisClass.up(I)J
--> [app] com.sirisuk.AnalysisClass.down(I)J
--> [lib] java.io.PrintStream.print(Ljava/lang/String;)V
--> [lib] java.io.PrintStream.println(Ljava/lang/String;)V

[com.sirisuk.AnalysisClass.<clinit>()V]

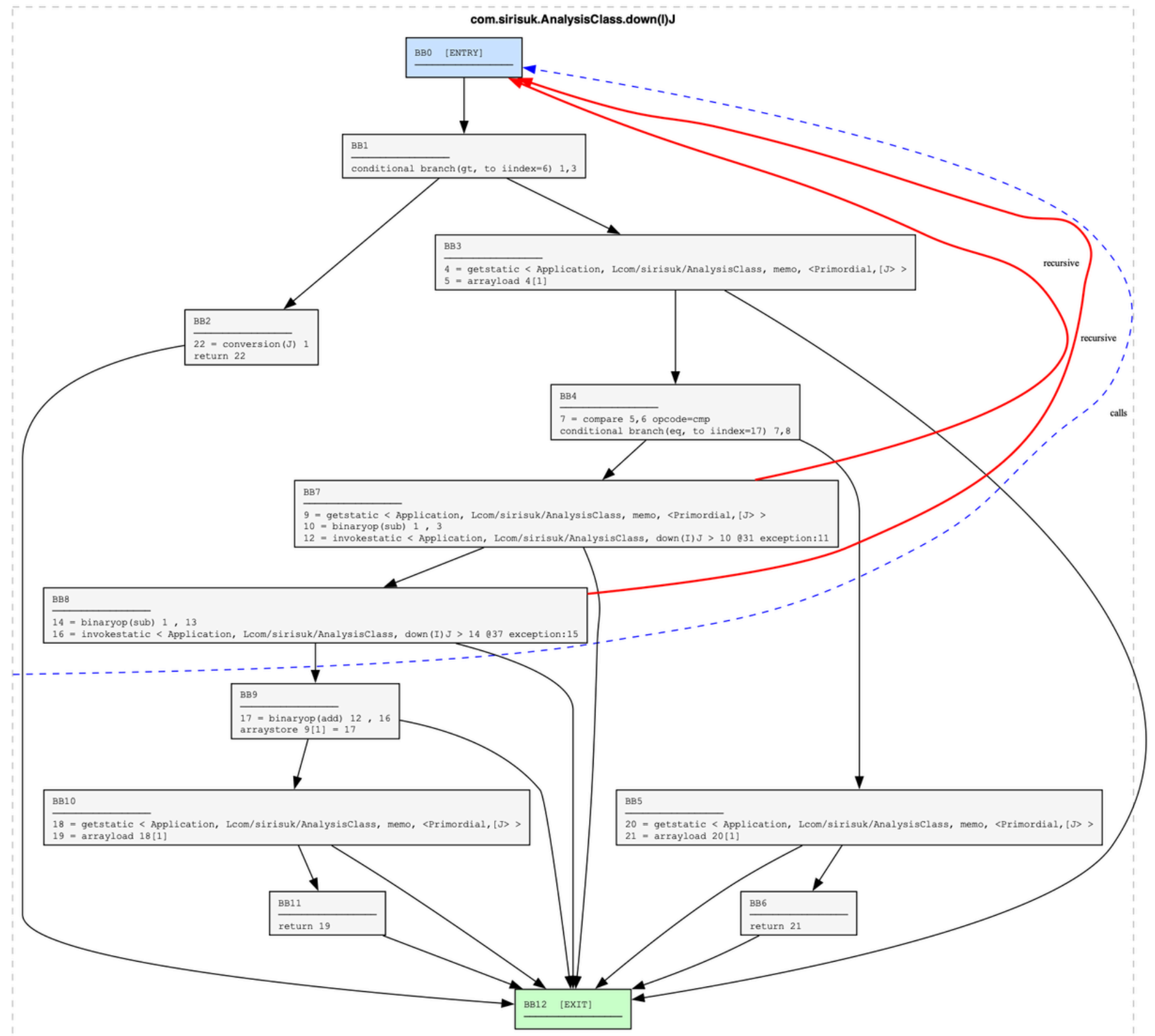
[com.sirisuk.AnalysisClass.up(I)J]

[com.sirisuk.AnalysisClass.down(I)J]
--> [app] com.sirisuk.AnalysisClass.down(I)J
```

The Visualize CFG

by making the BasicBlock as vertex and edges to successor node then I can build the CFG.

each BasicBlock (BB) have instructions based on the program implementation but represent as Intermediate Representation form (WALA IR)



the CFG for some methods

The CFG Problems

1. It's ***hard to read*** WALA IR (this is too low level and not human friendly)

BB6

```
13 = getstatic < Application, Ljava/lang/System, out, <Application,Ljava/io/PrintStream> >  
15 = invokestatic < Application, Lcom/sirisuk/AnalysisClass, down(I)J > 2 @34 exception:14
```

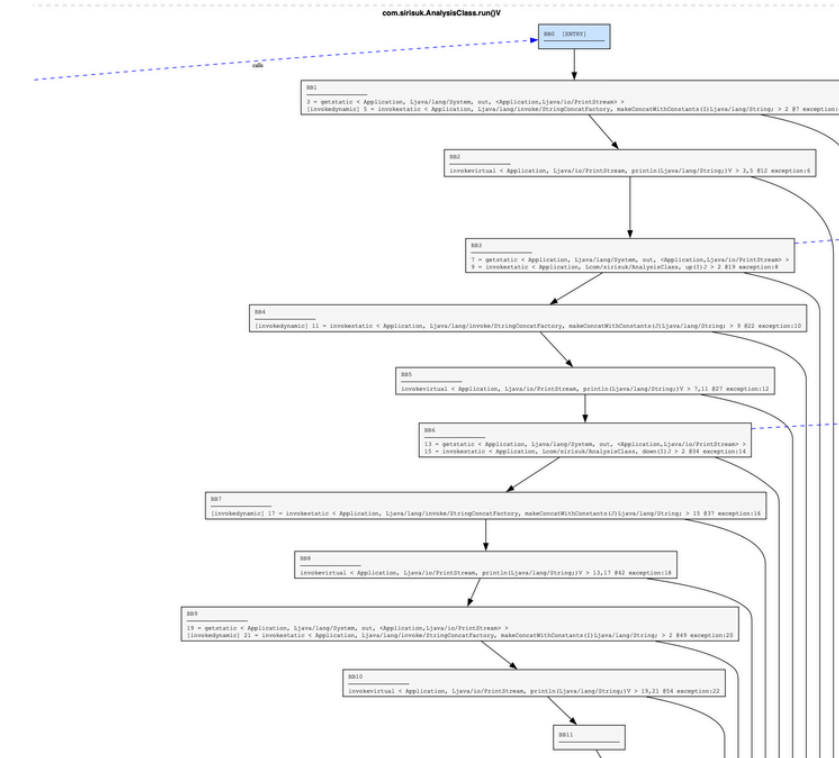
2. The same program flow *instruction not correctly show in the same BasicBlock*

```
public static void run() { 1 usage
    int n = 10;

    System.out.println("Fibonacci DP Demo (n = " + n + ")");
    System.out.println("Bottom-up : " + up(n));
    System.out.println("Top-down : " + down(n));

    System.out.println("\nSequence (0.." + n + "):");
    for (int i = 0; i <= n; i++) {
        System.out.print(up(i) + (i < n ? " " : "\n"));
    }

    testLoop();
}
```



the single BasicBlock instructions represent into mutliple BB node

FIX 1

to fix the first problem I will make conversion from **WALA IR into Java Source code** but WALA instruction index not same with ByteCode. then I have to know the relation between them.

WALA IR relation to ByteCode

I dicoverd that the ByteCode index bases on the instruction size as the offset and can be map back to source line number using WALA's IMethod class

- `getLineNumber()`

```
1  javap -c -p ../fibo/target/classes/com/sirisuk/AnalysisClass.class > fibo_class-lookup.txt
2  OUTPUT
3
4  ⚡
5  Compiled from "AnalysisClass.java"
6  public class com.sirisuk.AnalysisClass {
7
8      private static long[] memo;
9
10     public com.sirisuk.AnalysisClass();
11     Code:
12         0: aload_0
13         1: invokespecial #1
14         4: return
15
16     public static long up(int);
17     Code:
18         0: iload_0
19         1: iconst_1
20         2: if_icmpgt 8
21         5: iload_0
22         6: i2l
23         7: lreturn
24         8: iload_0
25         9: iconst_1
26        10: iadd
27        11: newarray      long
```

© com.ibm.wala.classLoader.IMethod

public abstract int `getLineNumber(int bcIndex)`

Returns: the source line number corresponding to a particular bytecode index, or -1 if the information is not available.

Gradle: com.ibm.wala:com.ib...1.7.2.jar

Transform WALA IR into JAVA

Since WALA IR not having same index based as the ByteCode then we need to map back using WALA's IByteCodeMethod class

- getByteCodeIndex()

```
© com.ibm.wala.classLoader.IBytecodeMethod<I>
public abstract int getBytecodeIndex(
    int i
)
throws com.ibm.wala.shrike.shrikeCT.InvalidClassFileException

Returns: the bytecode index corresponding to instruction i in the
        getInstructions() array

Throws: InvalidClassFileException

Gradle: com.ibm.wala:com.ibm.wala.core:1.7.2 (com.ibm.wala.core-1.7.2.jar)
```

Then simple mapping index code will look like this

```
int bcIndex = bytecodeMethod.getBytecodeIndex(instructionIndex); // SSA iindex → bytecode offset
int line    = method.getLineNumber(bcIndex);                  // bytecode offset → source line
```



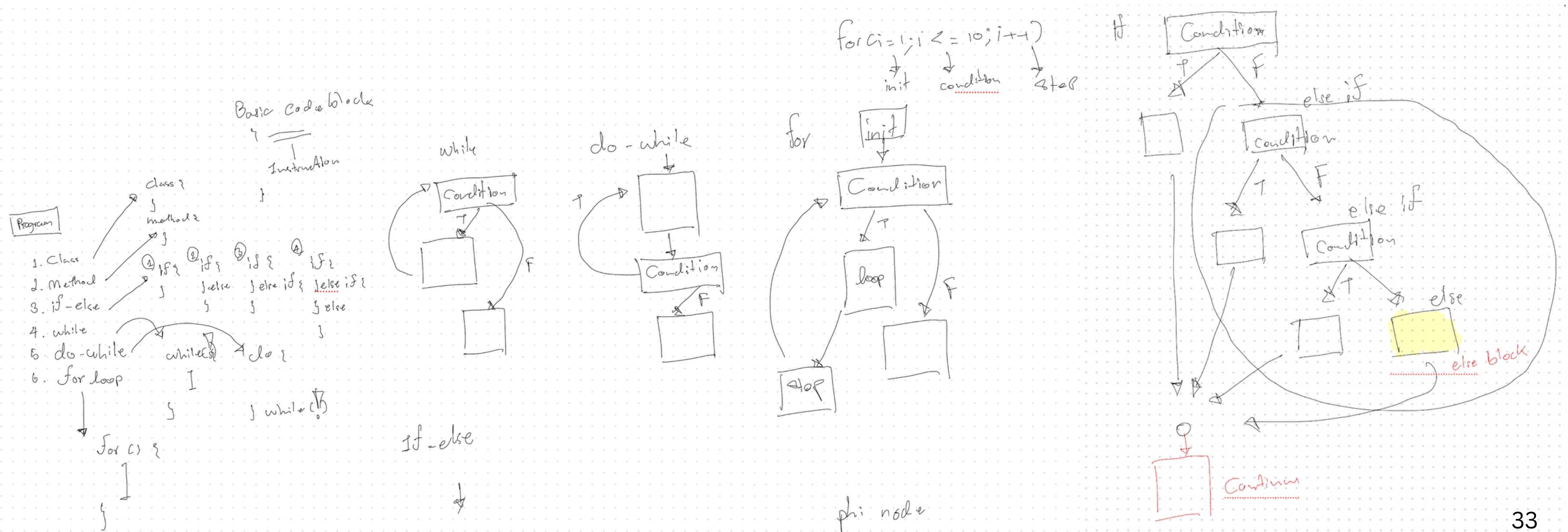
FIX 2

to fix the second problem I will ***perform BasicBlock merging*** that's to merge BasicBlock that have same flow or scope into one BasicBlock.

BUT HOW?

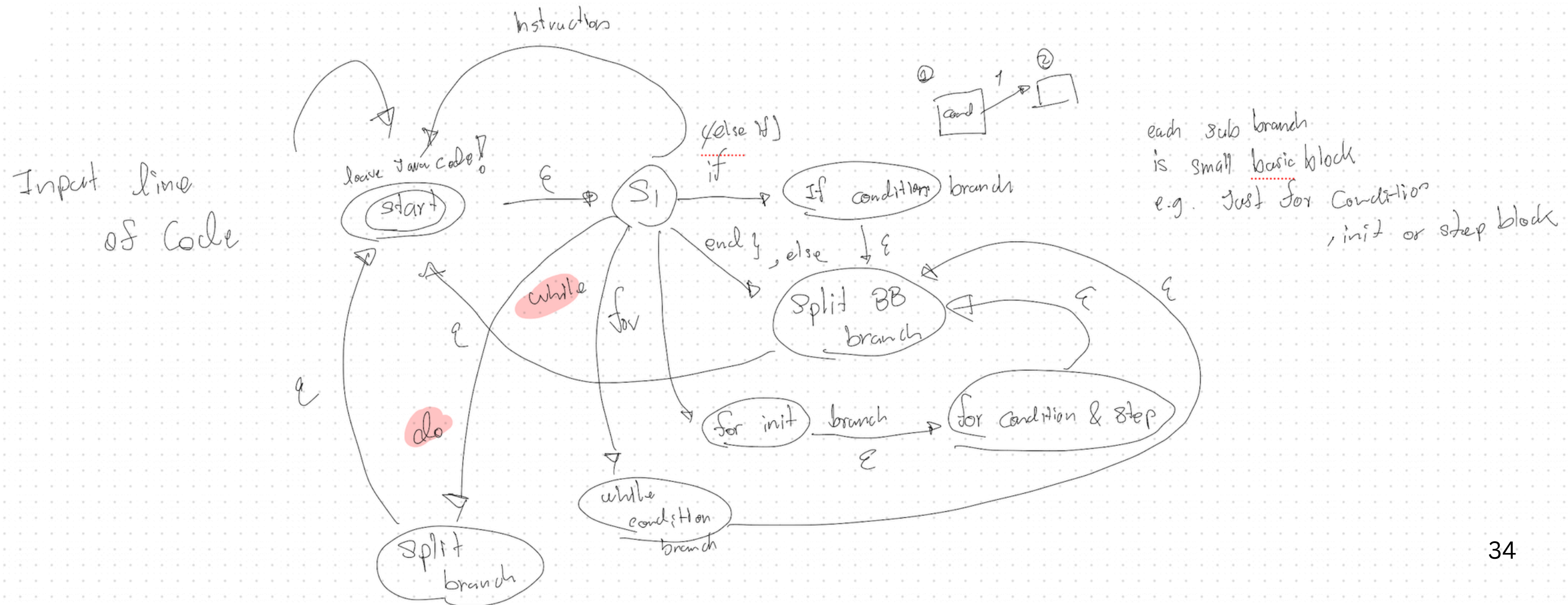
BasicBlock merging Algorithm

to make merge algorithm I have tried to visualize the language's control-flow into diagram for all possible form of BasicBlock



BasicBlock merging Algorithm cont.

I have an idea for detecting the control-flow and try to write down the NFA of program that take the source code line input and process to get split BB.



BasicBlock merging Algorithm cont.

benefit from WALA IR convertor into JAVA Source Code make controll-flow detection feasible because all BasicBlock have already been converted

by using regular expression

I can classify the ENUM as kinds of controll-flow or Instruction.

```
private static Kind classify(String text) { 1 usage 2 punsnx
    if (text == null) return Kind.INSTR;
    String t = text.strip();
    if (t.matches(regex: "^}\\s*else\\s+if\\s*\\((.*)") return Kind.ELSEIF;
    if (t.matches(regex: "^}\\s*else\\s+b.*") return Kind.ELSE;
    if (t.matches(regex: "^if\\s*\\((.*)") return Kind.IF;
    if (t.matches(regex: "^}\\s*while\\s*\\((.*)") return Kind.WHILE; // \\}\\s*?
    if (t.matches(regex: "^for\\s*\\((.*)") return Kind.FOR;
    if (t.matches(regex: "^do\\s+b.*") return Kind.DO;
    return Kind.INSTR;
}
```

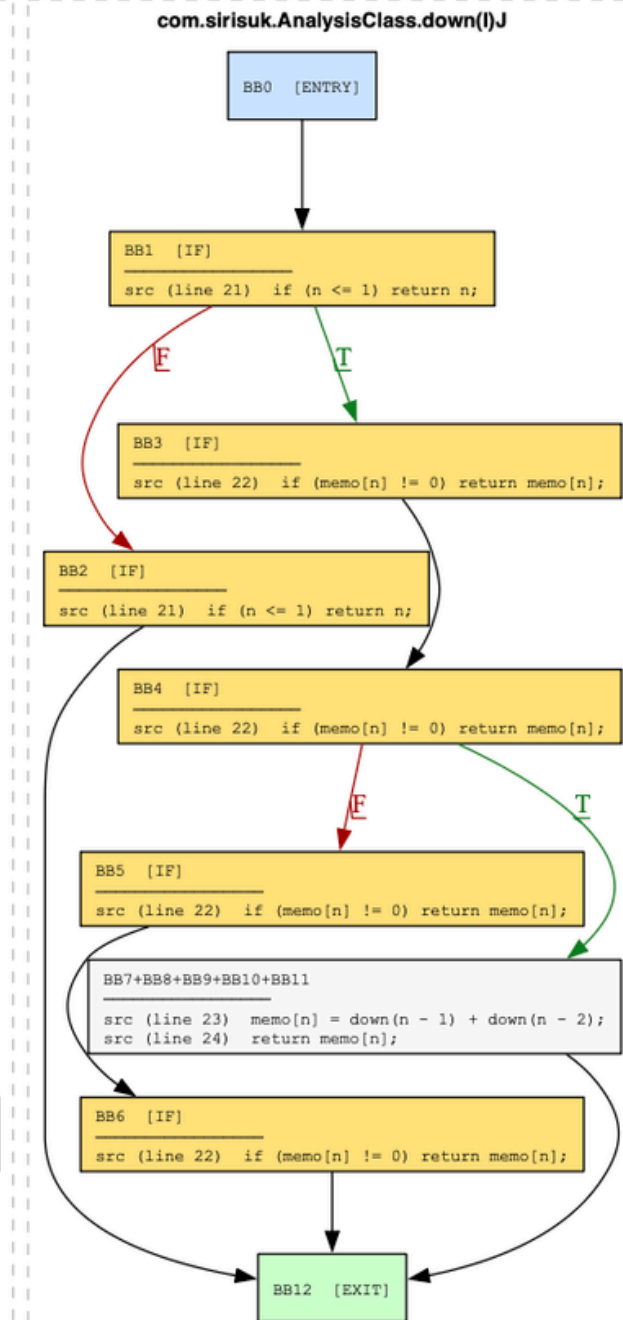
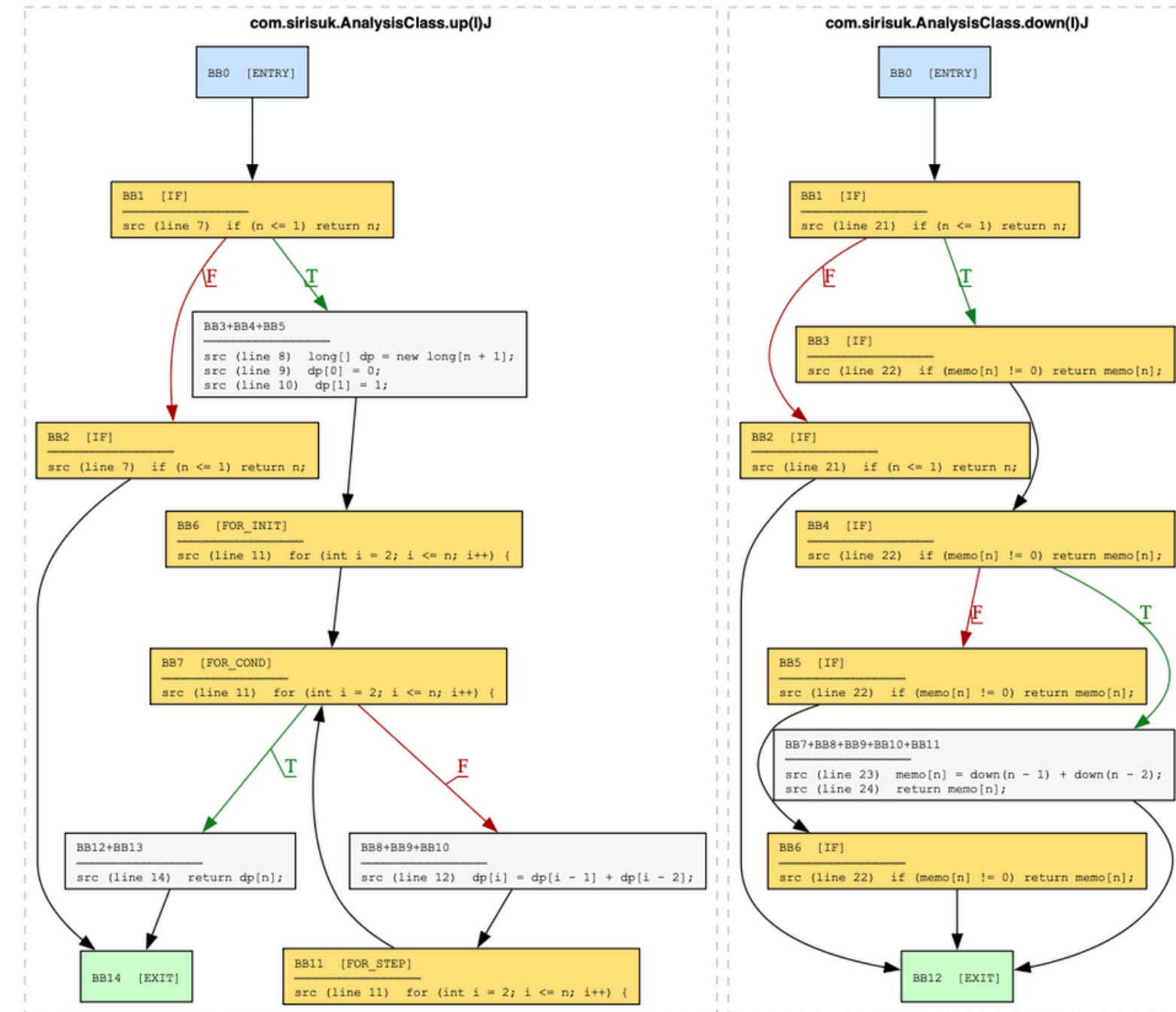
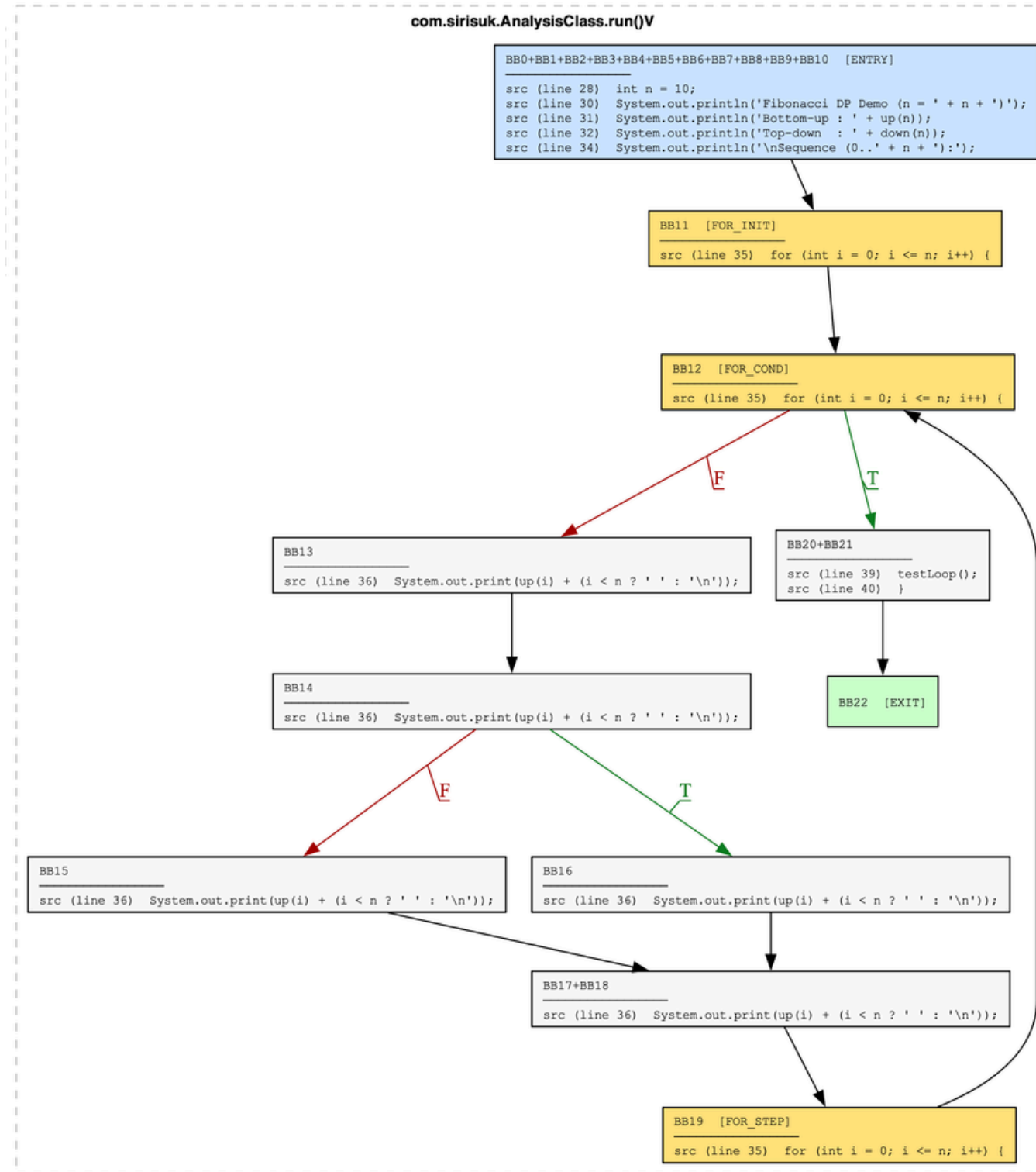
BasicBlock merging Algorithm cont.

And the main part is the rule for the algorithm to verify that the BasicBlock should merge or split.

- I have write down the rule to satisfy BasicBlock properties
- By looking and threat BasicBlock as just the node(vertex) and edges
- Now everynode have in-coming and out-coming edges
- So I use this properties along with the previos classification to verify correctness before merge

#	Rule	Code check	Fails when...	Idea group
1	Not a boundary block	<code>isEntryBlock() OR isExitBlock() OR isCatchBlock()</code>	block is ENTRY/EXIT/CATCH	(a) Structural isolation
2	Doesn't itself branch	<code>getNormalSuccessors(bb).size() > 1</code>	block ends in a decision (2+ outgoing edges)	(a) Structural isolation
3	Isn't itself a keyword block	<code>kindOf.get(bb.getNumber()) != Kind.INSTR</code>	block's source line is <code>if / for / while</code> /etc. — it's a header	(b) NFA keyword override
4	Exactly one predecessor	<code>preds.size() != 1</code>	block is a join point (2+ incoming edges)	(c) Clean single parent
5	Predecessor doesn't branch	<code>getNormalSuccessors(pred).size() != 1</code>	block is one arm of its parent's branch	(c) Clean single parent
6	Predecessor isn't a keyword block	<code>kindOf.get(pred.getNumber()) != Kind.INSTR</code>	parent is a closed <code>for / if</code> /etc. header — never glue onto it	(c) Clean single parent

CFG tools program



Summarize and learning outcomes

during past 3 months I have done team project to discover usage for the WALA framework and also be the project manager, do the static techniques research and develop CFG tools with new code translator program, design & implement BasicBlock merging algorithm.

Learning outcomes

- learn static techniques and how static analysis locate in compiler
- understand how backend process of JVM and WALA topologies
- got new techniques e.g the ByteCode relation index mapping, using new package manager like gradle, idea for utilize unix tools for testing CI/CD
- work with teams and use Agile in real world project
- learn that having pace on myself and spend time focus deep drive into work cause me understand better instead of try to work fast

Further Improvement

project limitation to fix

- the CFG tools not work well with in-line multi instruction
- only working with common control-flow keyword (if,else,while,for)
- there is a logic bug from reading boolean successor due to WALA IR operator have been swap

Future improvement

- fix some of the CFG tool limitation
- bring these static analysis knowledge to use with my senior project by checking the opensource project bugs or optimization approach and do the improvement

Thank you!

Checkout the project source code here
https://git.fim.uni-passau.de/rezaalipour/intern26_se1_sirisuk





<https://kaset.s.art/D70M7L>

Post working review
for supervisor
(form has been translated to English)

ข้อมูลทั่วไป/Work Term Information
ชื่อ-นามสกุล (Student's name - Lastname) *
Sirisuk Tharntham
รหัสประจำตัว (Student ID) *
6610402230